A DH-Transformation-Based Neural Network Architecture for Forward Kinematics Learning

Leandro de Souza Rosa

***Abstract*—...**

***Index Terms*—A, B, C.**

I. Introduction

Forward kinematics is arguably one of the most basic computations in robotics, which provides means of computing the position of joints, or other links, in a chosen reference frame (often the world frame). For industrial and commercial robots, the forward kinematics is computed based on a detailed and accurate system description provided by the manufacturer.

Nevertheless, for low-cost or in-house-built robots, achieving a satisfactory forward kinematics is not a trivial task, and often learning-based methods are adopted. (citation needed)

In order to learn forward kinematics, common approaches found in the literature often rely on using a monolithic Neural Network (NN) to approximate the mapping between joints and position, without giving much attention to the NN architecture and its impacts in learning efficiency and final model precision. Furthermore, little attention has been given for the learning scalability as the number of joints increase.

In this paper we propose a novel NN architecture based on Denavit-Hartenberg (DH) transformations for learning forward kinematics. The proposed architecture consists of one NN for estimating the transformation of each joint with respect to the previous one, and leverages the chaining properties of Deep Learning (DL) transformations for computing the forward kinematics with respect to a common reference frame.

In order to test the proposed architecture, we simulate robotic manipulators with 2, up to 7 joints, showing increasing gains in sampling efficiency and accuracy as the number of joints increase. We further demonstrate the impacts of using the proposed architecture on higher level computations based on the forward kinematics, namely velocity estimation through Jacobian computation and on a Reinforcement Learning (RL) policy learning.

II. Related Works

[1] probably the first proposal of learning forward kinematics with NN. Uses a monolithic network and a planar 2-link robotic arm.

[2] proposes to learn forward kinematics as a monolithic Gaussian Mixture Model (GMM) model. They test on a real robot with (3?)-Degree of Freedom (DoF).

[3] proposed to learn the forward and inverse kinematics together for parallel robots. The approach uses a monolithic model, which is learned online using a Locally Weighted Projection Regression (LWPR).

[4] proposes to solve the forward kinematics using Support Vector Regression (SVR).

[5] proposes to learn a policy with deep-Q learning. They use simulation and robot farms. (fells like I've the images of this paper in another paper. double check)

[6] presents as analysis on sample efficiency for learning the forward kinematics of linked robots. Using tailored parameterized models is efficient, other methods are not sample efficient. They focus on Fourier-Domain representations.

[7] solves the forward kinematics of a 6-DoF serial manipulator using a monolithic NN.

[8] study the influence of joint-position representations on learning forward kinematics with a monolithic NN for a 6-DoF robotic arm and a cylindrical robot. They test `Euler angles`, `vectorial`, `quaternions`, and `angle-axis` representation. Results favor quaternion representation.

[9] proposes learning the inverse kinematics and the dynamics model of serial robot manipulators using Generative Adversarial Networks (GANs).

[10] evaluates different function approximators for learning monolithic forward kinematic models. NNs perform the best.

[11] uses Gaussian Process Regression (GPR) for learning the forward kinematics based on vision. The approach considers a monolithic Gaussian Process (GP).

[12] uses a monolithic NN for learning forward kinematics for a 6-DoF robot manipulator in simulation. The learned model is transferred to a real robot (using the same training signals). Results show that the real robot achieves higher accuracy.

[13] seems to be the same as [12].

[14] uses a monolithic NN for learning forward and inverse kinematics of a 5-DoF robotic arm (check it out it seems they do it for 2 DoFs).

[15] basically lists methods for solving forward kinematics. (does not really add anything)

[16] uses a monolithic NN for learning forward kinematics of a 3-DoF robotic arm.

III. Preliminaries

Consider the serial robot manipulator illustrated in Figure 1 with j links and $j+1$ joints. The reference frames of the world and of J_0 are fixed and coincide.

Our task is to estimate the Cartesian position of each joint $\mathrm{J}_i | i \in \{1, \ldots, j\}$ in the world reference frame ($\mathbf{x}$) as a function of the rotation angles of $\mathrm{J}_i | i \in \{0, \ldots, j-1\}$ ($q_i | i \in \{0, \ldots, j-1\}$), as described in Equation 1. The Cartesian pose of each joint J_i is a d dimensional array (usually $n = 2, 3$), resulting in the estimation of $n \times j$ functions in total.

0000–0000/00$00.00 © 2021 IEEE

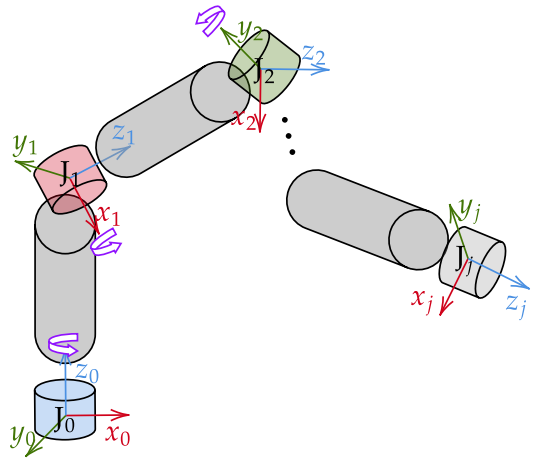

Fig. 1. A serial manipulator with j axes and $j+1$ joints J_i. Each axis (gray) is connected by a joint with a Cartesian reference frame (red, gree, blue) and can rotate over an arbitrary axis (purple). We consider, without loss of generality, the position of J_0 fixed at the world origin, and J_j to be the end-effector, which does not rotate. Elements are colored according to their rotation axis if they rotate, otherwise in gray. The Cartesian position of joint $\mathrm{J}_i | i \in \{1, \dots, j\}$ ($\mathbf{x}_i$) in the reference frame of J_{i-1} depends only on q_{i-1}.

$$\tilde{\mathbf{x}}(\mathbf{q}) = \left[\begin{array}{cccc} \tilde{\mathbf{x}}_1(q_0) & \tilde{\mathbf{x}}_2(q_0, q_1) & \dots & \tilde{\mathbf{x}}_j(q_0, \dots, q_{j-1}) \end{array} \right] \quad (1)$$

The function $\mathbf{x}(\mathbf{q})$ can be expressed using DH transformation matrices [17]. For a given J_i, the a transformation between its reference frame and the one of J_{i-1} is described by 2, where $\mathbf{T}$ and $\mathbf{D}$ are the rotation and displacement matrices, respectively.

$$\mathbf{T}^i_{i-1} = \left[\begin{array}{cc} \mathbf{R}^i_{i-1}(q_{i-1}) & \mathbf{D}^i_{i-1}(q_{i-1}) \\ \mathbf{0}_{1\times 3} & 1 \end{array} \right] \quad (2)$$

For a serial manipulator, the transformations can be chained for obtaining a transformation for any given joint J_i in the world reference frame, according to Equation 3.

$$\mathbf{T}^i_0 = \prod_{k=1}^{i} \mathbf{T}^k_{k-1} \mid i \in \{1, \dots, j\} \quad (3)$$

IV. Proposed Architecture

A. Overview

While approaches in the literature propose to learn a monolithic approximator $\tilde{\mathbf{x}}^m = \Phi^m(\mathbf{q}) : \mathbb{R}^j \rightarrow \mathbb{R}^{n\times j}$. Our main idea is to learn one approximator for each joint J_i, which encodes $\mathbf{T}^i_{i-1}$, and thus learning j predictors $\tilde{\mathbf{x}}^l_i = \Phi^l_i(\mathbf{q_i}) : \mathbb{R}^1 \rightarrow \mathbb{R}^{n\times(n+1)} \mid i \in \{1, \dots j\}$, and joining all Φ^l_i leads to the proposed model Φ^l. Figure 2 presents the proposed architecture's overview.

We implement Φ as NNs with l_n fully-connected layers, and l_s neurons for each layer. Each layer, except for the last, is followed by a `TanH` activation layer. While other approximators could be used, e.g., GPs as in [11], their evaluation falls out-of-the-scope of this paper.

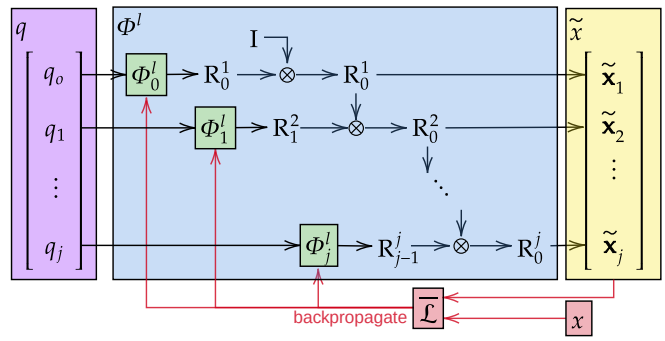

Fig. 2. Proposed architecture. For each J_i $i \in \{1, \dots, j\}$, a network Φ^l_i (green) learns $\mathbf{R}^i_{i-1}$, which is the DH transformation from J_{i-1} to J_i in function of q_i (purple). $\mathbf{R}^i_{i-1}$ is multiplied by the cumulative transformation of all previous joints, resulting in $\mathbf{R}^i_0$, from which the Cartesian position $\tilde{\mathbf{x}}_i$ (yellow) is obtained directly. Training is done using a loss $\mathcal{L}$ base on the Cartesian distance between the estimated and real values (red) as described in Section IV-B.

B. Loss Function

For training Φ, we follow the approaches in the literature (citation needed)and use the average Euclidean distance between the joints and their respective prediction, as described in Equation 4. Note that some approaches in the literature consider only the end-effector position and its prediction, which is reasonable since in a real robot, the position of individual joints is hard to obtain, while the end-effector's position can be obtained with off-the-shelf trackers and cameras (citation). In contrast, since we perform the training in simulation, we have the flexibility of considering all joints, leading to a richer loss signal, improving the training convergence.

$$\mathcal{L} = \frac{1}{j} \sum_{k=1}^{j} ||\tilde{x}_k - \mathbf{x}_k|| \quad (4)$$

C. Training Procedure

Since in this stage we use simulation, both Φ^m and Φ^l are trained in a straight-forward fashion according to Algorithm 1. For each epoch of training, $nSamples$ random poses are sampled in joint space (line 3), and the respective Cartesian position of each joint is obtained directly from the simulator (line 4) and added to a mem memory buffer (line 5). The size of mem is limited, and upon additions further than its limit, it drops a percentage of its oldest data (we use a 10% data drop in all experiments) for reducing data allocation overheads. Next, $nIter$ iterations of training are done with the data within mem by randomly sampling bs samples from the memory (line 8), computing the average loss (lines 9 to 14), which is then back-propagated through the network (line 15) with an lr learning rate.

It is worthy to notice that in our tests, we observed that randomly sampling poses leads to a much higher efficiency than performing random walks, which is to be expected due to the better state space coverage. However, when considering a real robot in the RL loop, it is unpractical to sample the state space this way, since the robot will be naturally performing a

Algorithm 1: Training procedure for Φ.

```
for e ∈ {1, ..., epochs} do
    for r ∈ {1, ..., nSamples} do
        q ← random joint positions;
        x ← Cartesian position of all joints given q;
        add (q, x) to mem;
    end
    for t ∈ {1, ..., nIter} do
        (qs, xs) ← get bs random samples from mem;
        L̄ ← 0;
        for (q, x) ∈ (qs, xs) do
            L̄ ← L̄ + L;
        end
    end
    L̄ ← L̄/nIter;
    backpropagate(L̄, lr) through Φ;
end
```

random walk. In section VI, we present a RL case of study highlighting the proposed architecture impacts in this scope.

V. Results

A. Testing Environment

We build upon the `Reacher-v4` environment from OpenAI `Gym` [18], which consists of a 2D serial manipulator with 2 joints simulated using `MuJoCo` [19]. In order to evaluate the proposed approach for more realistic robots, we extend the environment enabling for 3D, and adding models for robots up to 7 joints[1].

The Φ modules are implemented using `PyTorch` [20], allowing for integration with `Gym`, and the derivatives used in Section V-C are computed using `PyTorch`'s `Autograd` [21].

For the sake of briefness we restrict the results presented in this section for the 3D case, as robots with more than 3 joints usually act in the three dimensional space.

B. Parameters Tuning

In order to assure appropriate hyper-parameters for Φ, we use `Ray Tune` [22] with `Optuna` [23] as search engine and a `Asynchronous Hyper-Band` [24] as scheduler. The hyper-parameters are sampled in the interval presented in Table I, We empirically found that $nSamples = 100$ and $nIter = 25$ are appropriated values tuning hyper-parameters for Algorithm 1.

During the tuning process, the average loss over 10 learning trials $\bar{\mathcal{L}}^{tune}$, $epochs$, and the total number of parameters in Φ ($|\Phi|$) are passed to `Optuna` every 50 $epochs$ until $\bar{\mathcal{L}}^{tune} <= 0.005$ or $epochs = 2000$ are reached. Optimizing $\bar{\mathcal{L}}^{tune}$ leads for hyper-parameters which allow for effective learning the forward kinematics; optimizing $epochs$ incentivize the optimizer finding hyper-parameters that improve the learning sampling-efficiency; and optimizing $|\Phi|$ gives priority for smaller networks.

[1]Publicly available source code: https://github.com/lsrosa/kinematics_learning

TABLE I
Φ HYPER-PARAMETERS SEARCH SPACE. "LOG UNIFORM" SAMPLING CHOOSES A RANDOM NUMBER IN AN UNIFORM DISTRIBUTION IN THE LOG OF THE GIVEN RANGE, "CATEGORICAL" SAMPLING CHOOSES A RANDOM VALUE FROM THE RANGE LIST.

hyper-parameter	sampling	range
lr	log uniform	$[10^{-5}, 10^{-3}]$
l_n	categorical	$[2, 3, 4, 5, 6, 7, 8]$
l_s	categorical	$[8, 16, 32, 64, 128]$
bs	categorical	$[8, 16, 32, 64, 128]$

A total of 50 hyper-parameters samples are computed and collected by `Ray Tune`, and the final values are given by the sample with smallest $\bar{\mathcal{L}}^{tune}$ among the Pareto dominating samples according to $\bar{\mathcal{L}}^{tune}$, $epochs$, and $|\Phi|$. Table II presents the hyper-parameters found for Φ^m and Φ^l for robots with $j = \{2, \ldots, 7\}$. The found hyper-parameters values demonstrate the consequences of using a monolithic architecture: the complexity of the function being approximated grows with j (as described in Equation 1), and hence a larger NN is more suitable. In comparison, the function being approximated with the proposed architecture is constant in j, and increasing j increases the number of approximators linearly. As result, the proposed architecture is expected to scale better with j than its monolithic counterpart, both in precision and sample efficiency.

TABLE II
HYPER-PARAMETERS FOUND FOR ROBOTS WITH VARYING NUMBER OF JOINTS.

$n = 2$							
Φ^l	lr	a	a	a	a	a	a
	l_n	a	a	a	a	a	a
	l_s	a	a	a	a	a	a
	bs	a	a	a	a	a	a
	$\lvert\Phi\rvert$	a	a	a	a	a	a
$n = 3$							
Φ^l	lr	a	a	a	a	a	a
	l_n	a	a	a	a	a	a
	l_s	a	a	a	a	a	a
	bs	a	a	a	a	a	a
	$\lvert\Phi\rvert$	a	a	a	a	a	a

C. Training, Loss, and Accuracy Results

The models are trained with the hyper-parameters found according to Section V-B. The training budget is set as 2000 epochs with $nSamples = 100$ and $nIter = 25$.

Figure 3 presents the losses when learning Φ^m and Φ^l, which shows that the proposed architecture outperforms the monolithic approach found used in the literature.

For qualitative comparison, Figure 4 presents the joints position estimation obtained by Φ^m and Φ^l for $j = 7$ only (other joint numbers cases are omitted for briefness), in which the higher accuracy can be seem.

Figure 5 presents the joint prediction error for 1000 samples for the robot with $j = 7$, showing that the Φ^l achieves a higher accuracy than Φ^m, which is expected due to the improved learning efficiency shown in Figure 3.

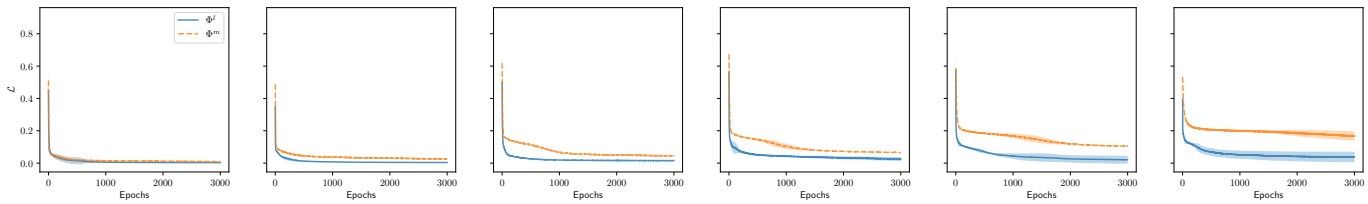

Fig. 3. Loss comparison between Φ^m and Φ^l. The solid line and shaded area represent the mean loss and the area between minimum and maximum losses over 30 trials.

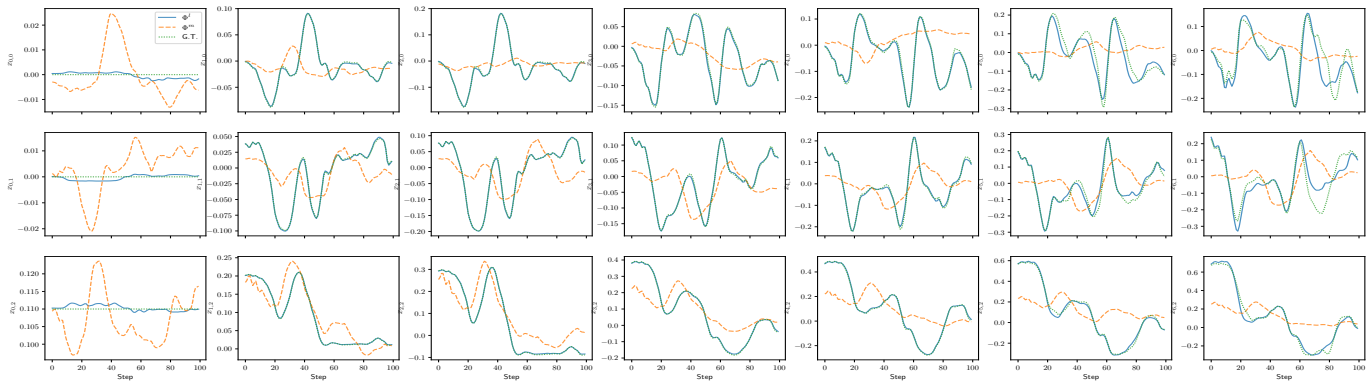

Fig. 4. Joints position estimation comparison for Φ^m and Φ^l for 100 steps.

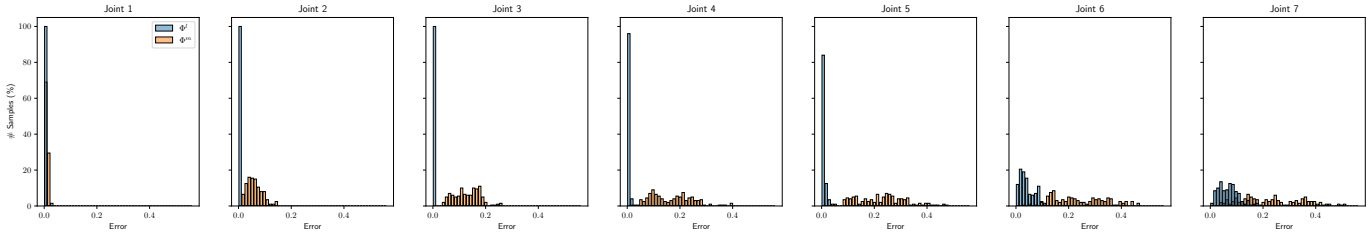

Fig. 5. Accuracy comparison for Φ^m and Φ^l.

We can observe a characteristic for the models accuracies: the error increases for the further joints. For Φ^l, this results from the error propagation from one joint to another since estimation from the previous joint is used for computing the following joint's estimation; for Φ^m the increasing error is lead by the fact that the function being approximated for the latter joints is more complex (see Equation 1). It is important to emphasize that using a monolithic architecture for estimating the forward kinematics is inappropriate, as the NN outputs are related to functions with varying complexity, leading non-uniform learning in the network, which is prone to fall into local-minima. In fact, we speculate that the results presented in Figure 3 for Φ^m are explained by such local-minima: once the loss drops due to sufficient accuracy on learning the first joints, the optimization process struggles to refine the parameters, leading to the very slowly decaying loss observed after an initial sudden drop.

Next, to demonstrate the accuracy impacts, we estimate the end-effector velocity $\dot{x}_j = [\mathbf{J}\dot{q}]_j$, where $\mathbf{J}$ is the unknown Jacobian matrix of the robot, as $\tilde{\dot{x}}_j = \left[\frac{\partial \Phi}{\partial q}\dot{q}\right]_j$. That is, we use the learned network for estimating the unknown Jacobian of the model, which has its well-known applications in inverse kinematics [25] computation, Cartesian impedance controllers [26], and tracking controllers [27]. Figure 6 shows the velocity estimation and ground-truth obtained from the simulator for $n = 3$ and $j = 7$. The better accuracy of the proposed architecture indicates its adequacy for being used in higher level applications, while the monolithic architecture poor accuracy might impact higher level applications badly.

VI. Reinforcement Learning Case of Study

This section presents a case of study using the forward kinematics estimation in a RL loop, which consists of two steps. In the fist step, we train Φ and a policy $\Pi : \mathbb{R}^{n\times j} \rightarrow \mathbb{R}^j$, which given the Cartesian position of the robot joints, learns the torques for each joint for moving the end-effector towards the target position, using the pre-trained Φ model learned in Section V for estimating $\mathbf{x} = \Phi(\mathbf{q})$. In the second step, we create a different robot with the same number of joints, by varying its links' dimension according to a normal distribution $\mathcal{N}(\mathbf{d}, \sigma = 0.01)$, where $\mathbf{d}$ is the dimensions of the robot used in the first step, and retrain both Φ and Π on the new robot.

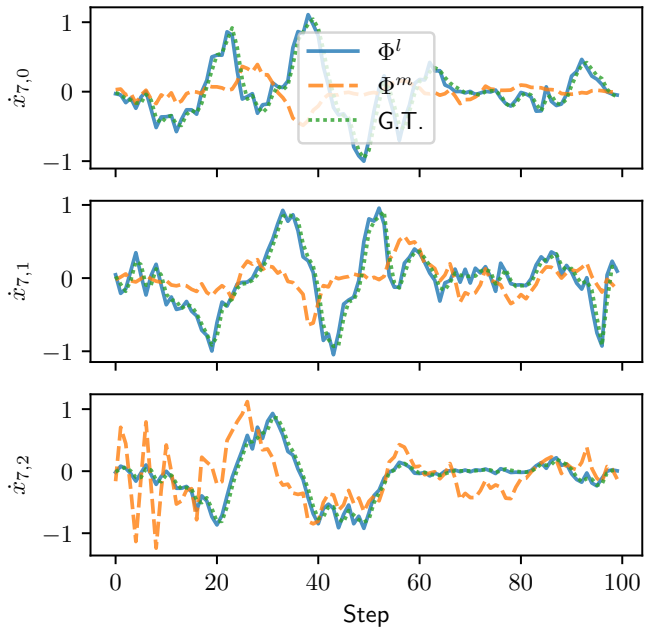

Fig. 6. End-effector velocity estimation comparison for Φ^m and Φ^l.

As such, Π encompasses learning the robot's dynamics, and control, which is a considerably hard task for high-dimensions.

This study case mimics a straight-forward transfer-learning case, in which first a model is firstly trained in simulation, and then refined in a real robot, which is slightly different from the simulated one. These differences may come from imprecise measures, or from an unknown model, as in the case of budget robot manipulators [28]. Furthermore, more elaborated transfer-learning approaches [29], [30] might also benefit from the proposed network, but its evaluation falls out-or-the-scope with this paper.

The results in this section are limited to the 3D and a robot with 7 joints, and the second part of the learning is also done in simulation.

For Π we use a `SAC` from `Stable-baselines3` [31], and Φ is introduced into the `SAC` as such its outputs are the inputs of `SAC`'s features extraction step. Figure 7 presents the rewards obtained in both steps, where we can see...

VII. Conclusions

Acknowledgements

This work was supported by ....

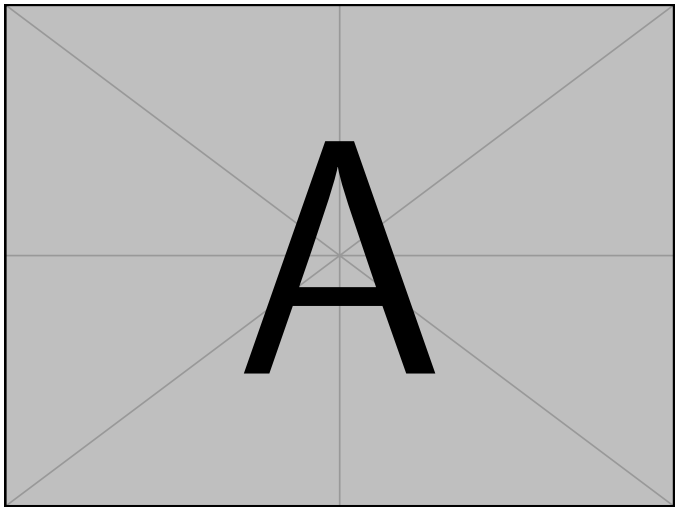

Fig. 7. Reward for learning Π using the pre-trained Φ, and for adapting Π to the modified robot using.

References

[1] L. Nguyen, R. Patel, and K. Khorasani, "Neural network architectures for the forward kinematics problem in robotics," in *1990 IJCNN International Joint Conference on Neural Networks*, 1990, pp. 393–399 vol.3.

[2] E. Gribovskaya, S. Khansari-Zadeh, and A. Billard, "Learning Nonlinear Multivariate Dynamics of Motion in Robotic Manipulators," *The International Journal of Robotics Research*, vol. 30, no. 1, pp. 80–117, 2011.

[3] B. Damas and J. Santos-Victor, "An online algorithm for simultaneously learning forward and inverse kinematics," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 1499–1506.

[4] A. Morell, M. Tarokh, and L. Acosta, "Solving the forward kinematics problem in parallel robots using Support Vector Regression," *Engineering Applications of Artificial Intelligence*, vol. 26, no. 7, pp. 1698–1706, 2013.

[5] S. Gu, E. Holly, T. Lillicrap, and S. Levine, "Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 3389–3396.

[6] D. Kubus, R. Rayyes, and J. J. Steil, "Learning Forward and Inverse Kinematics Maps Efficiently," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 5133–5140.

[7] M. Theofanidis, S. I. Sayed, J. Cloud, J. Brady, and F. Makedon, "Kinematic Estimation with Neural Networks for Robotic Manipulators," in *Artificial Neural Networks and Machine Learning – ICANN 2018*, V. Kůrková, Y. Manolopoulos, B. Hammer, L. Iliadis, and I. Maglogiannis, Eds. Cham: Springer International Publishing, 2018, pp. 795–802.

[8] R. M. Grassmann and J. Burgner-Kahrs, "On the Merits of Joint Space and Orientation Representations in Learning the Forward Kinematics in SE(3)," in *Proceedings of Robotics: Science and Systems*, Freiburgim-Breisgau, Germany, June 2019.

[9] H. Ren and P. Ben-Tzvi, "Learning inverse kinematics and dynamics of a robotic manipulator using generative adversarial networks," *Robotics and Autonomous Systems*, vol. 124, p. 103386, 2020.

[10] I. M. Loutfi, A. B. Boutchouang, A. Melingui, O. Lakhal, F. B. Motto, and R. Merzouki, "Learning-Based Approaches for Forward Kinematic Modeling of Continuum Manipulators," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 9899–9904, 2020, 21st IFAC World Congress.

[11] A. Dalla Libera, N. Castaman, S. Ghidoni, and R. Carli, "Autonomous Learning of the Robot Kinematic Model," *IEEE Transactions on Robotics*, vol. 37, no. 3, pp. 877–892, 2021.

[12] K. P. and B. B. Choudhury, "Transfer Learning-Based Artificial Neural Network for Forward Kinematic Estimation of 6-DOF Robot," *International Journal of Applied Metaheuristic Computing (IJAMC)*, vol. 13, no. 1, pp. 1–15, January 2022.

[13] P. Kesaba and B. Choudhury, "Forward Kinematic Analysis of 6-R Industrial Robot Using Active Sampled Neural Network Regressor," *International Journal of Software Innovation*, vol. 10, no. 1, pp. 1–14, 2022 2022.

[14] A.-N. Sharkawy, "Forward and inverse kinematics solution of a robotic manipulator using a multilayer feedforward neural network," *Journal of Mechanical and Energy Engineering*, vol. 6, no. 2, Oct. 2022.

[15] T. P. Valayil and R. S. Augustine, "Methods to solve forward kinematics of parallel and serial manipulators," in *AIP Conference Proceedings*, vol. 2670, no. 1. AIP Publishing, 2022.

[16] A.-N. Sharkawy and S. Khairullah, "Forward and Inverse Kinematics Solution of A 3-DOF Articulated Robotic Manipulator Using Artificial Neural Network," *International Journal of Robotics and Control Systems*, vol. 3, no. 2, pp. 330–353, 2023.

[17] S. Kucuk and Z. Bingul, *Robot Kinematics: Forward and Inverse Kinematics*, S. Cubero, Ed. InTech, 2006.

[18] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "OpenAI Gym," 2016.

[19] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2012, pp. 5026–5033.

[20] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds. Curran Associates, Inc., 2019, pp. 8024–8035.

[21] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in PyTorch," in *NIPS-W*, 2017.

[22] R. Liaw, E. Liang, R. Nishihara, P. Moritz, J. E. Gonzalez, and I. Stoica, "Tune: A Research Platform for Distributed Model Selection and Training," *arXiv preprint arXiv:1807.05118*, 2018.

[23] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

[24] L. Li, K. Jamieson, A. Rostamizadeh, K. Gonina, M. Hardt, B. Recht, and A. Talwalkar, "Massively Parallel Hyperparameter Tuning," 2018.

[25] K. J. Waldron, S.-L. Wang, and S. J. Bolin, "A Study of the Jacobian Matrix of Serial Manipulators," *Journal of Mechanisms, Transmissions, and Automation in Design*, vol. 107, no. 2, pp. 230–237, 06 1985.

[26] P. Sánchez-Sánchez and F. Reyes-Cortés, "Cartesian control for robot manipulators," *robot manipulators trends and development*, pp. 165–212, 2010.

[27] D. Chen, Y. Zhang, and S. Li, "Tracking Control of Robot Manipulators with Unknown Models: A Jacobian-Matrix-Adaption Method," *IEEE Transactions on Industrial Informatics*, vol. 14, no. 7, pp. 3044–3053, 2018.

[28] L. Čehovin Zajc, A. Rezelj, and D. Skočaj, "Low-Cost Open-Source Robotic Platform for Education," *IEEE Transactions on Learning Technologies*, vol. 16, no. 1, pp. 18–25, 2023.

[29] J. Stüber, M. Kopicki, and C. Zito, "Feature-Based Transfer Learning for Robotic Push Manipulation," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 5643–5650.

[30] B. Bócsi, L. Csató, and J. Peters, "Alignment-based transfer learning for robot models," in *The 2013 International Joint Conference on Neural Networks (IJCNN)*, 2013, pp. 1–7.

[31] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable Reinforcement Learning Implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.